World Machine Model

For Notepad++

Submitted for Software Engineering

World

1. The user has a computer running Microsoft Windows (XP or later).
2. The user has sufficient permissions to install software on the machine.
3. There is no requirement for an internet connection during normal editing operations.
4. The user works with text-based files such as source code, configuration files, scripts, or plain text documents.
5. The user may work with files encoded in various character sets (UTF-8, UTF-16, ANSI, etc.).
6. There is a file system accessible to the application for reading and writing documents.
7. The user may want to extend the editor's capabilities through third-party plugins.
8. Multiple files may need to be open and accessible at the same time within a single application window.
9. The user may need to process or edit large files (several megabytes or more) without performance degradation.
10. The user may work across multiple programming languages or file formats within the same editing session.
11. There is an open-source editing component called Scintilla available, which provides the core text rendering, cursor management, margin display, and undo/redo capabilities that the editor depends on.
12. There is an open-source lexer library called Lexilla available, which supplies individual language parser modules that can be coupled to Scintilla to enable syntax highlighting for a wide range of programming languages.
13. The user may want to manually change the active syntax language of a document at any time without closing or reopening the file.
14. The user may need to zoom in or zoom out on the text in the editor to improve readability depending on screen size or personal preference.
15. The user may want to quickly navigate back to previously opened files without browsing the file system again.
16. The user may need to mark specific lines in a document and return to them quickly during a long editing session.

Requirements

1. The editor should support opening, editing, saving, and closing multiple files simultaneously using a tabbed interface.
2. The editor should provide syntax highlighting for at least 80 programming languages, including C++, Python, JavaScript, HTML, CSS, SQL, and VHDL.
3. The editor should allow the user to search within a document using plain text or regular expressions (regex).
4. The editor should allow the user to perform find-and-replace operations across a single file or across multiple files in a directory.
5. The editor should display line numbers alongside the editing area.
6. The editor should support code folding so that the user can collapse and expand blocks of code.
7. The editor should provide auto-completion suggestions for keywords and previously typed words.
8. The editor should allow the user to record, save, and replay macros for automating repetitive editing tasks.
9. The editor should support a plugin system so that users can install, manage, and remove third-party extensions via a built-in Plugin Admin.
10. The editor should allow the user to open two views (split view) of the same or different documents side by side.
11. The editor should support multiple character encodings and allow the user to convert between them.
12. The editor should allow the user to customise the appearance of the interface, including themes, fonts, and colour schemes.
13. The editor should allow the user to define custom keyboard shortcuts for any available command.
14. The editor should be able to compare two files and highlight their differences.
15. The editor should support session management so that the user can save and restore a set of open documents.
16. The editor should allow the user to manually change the active language mode of any open document at any time via a dedicated Language menu.
17. The editor should allow the user to zoom in and zoom out on the displayed text using keyboard shortcuts or the mouse scroll wheel.
18. The editor should support word wrap so that long lines are displayed within the visible window width without requiring horizontal scrolling.
19. The editor should highlight matching brackets, parentheses, and braces when the cursor is placed next to one.
20. The editor should allow the user to convert the line endings of a document between Windows (CRLF), Unix (LF), and old Mac (CR) formats.
21. The editor should maintain a list of recently opened files so that the user can reopen them quickly from the File menu.
22. The editor should support bookmarks so that the user can mark specific lines and navigate between them with dedicated commands.
23. The editor should allow the user to jump directly to a specific line number in the active document.

24. The editor should provide a persistent toolbar giving single-click access to the most frequent commands, covering in order: New, Open, Save, Save All, Close, Close All, Print, Cut, Copy, Paste, Undo, Redo, Find, Replace, Zoom In, Zoom Out, Synchronize Vertical Scrolling, Synchronize Horizontal Scrolling, Word Wrap, Show All Characters, Define Your Language, Document Map, Document List, Function List, Folder as Workspace, Monitoring, Start Recording, Stop Recording, and Playback.
25. The File menu should provide: New document, Open, Open Containing Folder, Open Folder as Workspace, Reload from Disk, Save, Save As, Save a Copy As, Save All, Rename, Close, Close All, Close Multiple Documents, Move to Recycle Bin, Load Session, Save Session, Print, Print Now, a numbered recent files list, Restore Recent Closed File, Open All Recent Files, Empty Recent Files List, and Exit.
26. The Edit menu should provide: Undo, Redo, Cut, Copy, Paste, Delete, Select All, Begin/End Select, Begin/End Select in Column Mode, and submenus for Insert, Copy to Clipboard, Indent, Convert Case, Line Operations, Comment/Uncomment, Auto-Completion, EOL Conversion, Blank Operations, Paste Special, and On Selection, as well as Multi-select All, Multi-select Next, Column Mode, Column Editor, Character Panel, Clipboard History, and Read-Only toggling for both Notepad++ and Windows file attributes.
27. The Search menu should provide: Find, Find in Files, Find Next, Find Previous, Select and Find Next, Select and Find Previous, Volatile search variants, Replace, Incremental Search, Search Results Window, Go To (line/position), Go to Matching Brace, Select All In-between matching delimiters, Mark, Change History, Style All Occurrences of Token, Style One Token, Clear Style, Jump Up, Jump Down, Copy Styled Text, Bookmark navigation, and Find Characters in Range.
28. The View menu should provide: Always on Top, Toggle Full Screen Mode, Post-It, Distraction Free Mode, View Current File in external viewer, Show Symbol, Zoom, Move/Clone Current Document, Tab controls, Word Wrap, Focus on Another View, Hide Lines, Fold All, Unfold All, Fold Current Level, Unfold Current Level, Fold/Unfold by Level submenus, Summary, Project Panels, Folder as Workspace, Document Map, Document List, Function List, Synchronize Vertical Scrolling, Synchronize Horizontal Scrolling, Text Direction RTL/LTR, and Monitoring.
29. The Encoding menu should allow the user to set the active document's encoding to ANSI, UTF-8, UTF-8-BOM, UTF-16 BE BOM, or UTF-16 LE BOM, to access extended character sets, and to convert the current document between any of those encodings without losing its content.
30. The Language menu should list all supported programming languages organized alphabetically by first letter (A through Z), with individual named entries for XML, YAML, and KIXtart, as well as User Defined Language configuration, pre-installed Markdown (standard and dark mode), and a User-Defined option for custom language definitions.
31. The Settings menu should provide access to: Preferences (all general editor options), Style Configurator (per-language colour and font customisation), Shortcut Mapper (complete keyboard shortcut management), Settings Import, and the Popup Context Menu editor.
32. The Tools menu should provide built-in cryptographic hash computation for MD5, SHA-1, SHA-256, and SHA-512, operable on the selected text or the entire active document.

33. The Macro menu should provide: Start Recording, Stop Recording, Playback, Save Current Recorded Macro, Run a Macro Multiple Times, Trim Trailing Space and Save, and Modify Shortcut/Delete Macro.
34. The Run menu should allow the user to execute arbitrary external commands via the Run dialog (F5), provide configurable quick-launch entries and allow modification or deletion of any run command and its shortcut.
35. The Plugins menu should display a submenu for each installed plugin, and provide dedicated entries for Plugins Admin (browsing, installing, updating, and removing plugins from the online repository) and Open Plugins Folder (opening the local plugins directory in Windows Explorer).
36. The Window menu should allow the user to sort open documents by name, path, or type, open the Windows dialog listing all open files, and switch directly to any open document listed by filename.
37. The Help menu (?) should provide: Command Line Arguments documentation, links to the Notepad++ Home page, Project Page, Online User Manual, and Community Forum, the built-in software updater, Updater Proxy configuration, Debug Info dialog, and the About Notepad++ dialog.

Specification

1. The application will be built as a native Win32 desktop application using C++ and the Scintilla editing library.
2. Scintilla will handle all low-level editing responsibilities including text rendering, selection, undo/redo history, code folding margins, call tips, and visual indicators, exposing these capabilities to Notepad++ through a message-based API.
3. The application will consume no more than approximately 32 MB of RAM under typical usage conditions.
4. The tabbed document interface will allow the user to open an unlimited number of files, each accessible via a tab at the top of the editor window.
5. Syntax highlighting will be applied in real time as the user types, based on the file extension or a manually selected language mode.
6. Language lexers responsible for tokenising and colouring source code will be provided by Lexilla, a dedicated open-source lexer library linked into the application, allowing individual language modules to be updated or extended independently of the core Scintilla component.
7. The find-and-replace engine will support Perl-Compatible Regular Expressions (PCRE) for advanced pattern matching.
8. Code folding will be triggered by the margin gutter and will support folding by indentation level or language-specific block delimiters (e.g., {...} in C++).
9. Auto-completion will activate after a configurable number of characters and will draw from the current document's word list as well as language-specific keyword sets.
10. Macros will be recorded as sequences of keystrokes and editor commands, stored in an XML-based configuration file, and playable back with a single shortcut or menu action.
11. The Plugin Admin will connect to a curated online repository of plugins and will support install, update, and removal without restarting the application.

12. The document split-view feature will allow two independent edit panes within the same application window, each with its own scroll position and cursor.
13. The application will support encoding detection on file open and will allow manual conversion between UTF-8, UTF-16 LE/BE, UTF-8 BOM, and ANSI encodings.
14. The Style Configurator will allow per-language customisation of token colours, font faces, font sizes, and background colours, saved to an XML theme file.
15. Column/block selection will be available by holding the Alt key while selecting text with the mouse or keyboard, allowing simultaneous editing of multiple lines.
16. The application will provide a Document Map panel showing a zoomed-out overview of the full document for rapid navigation.
17. All user preferences, session data, keyboard shortcuts, and language settings will be stored in XML configuration files located in the application's installation directory or the user's AppData folder, supporting portable operation.
18. The active language mode of any open document will be selectable at any time via the Language menu, with the newly chosen Lexilla lexer applied immediately to the entire document without requiring the file to be closed or reopened.
19. Zoom level will be adjustable in discrete steps ranging from -10 to +20 using Ctrl+Mouse Wheel or Ctrl+Numpad Plus and Minus, and the current zoom level will be retained for the duration of the session.
20. Word wrap will be toggled via the View menu and will reflow all visible lines to fit the current editor pane width without modifying the underlying file content in any way.
21. The editor will highlight the matching opening or closing bracket, parenthesis, or brace in a configurable colour whenever the cursor is positioned immediately adjacent to one of these characters.
22. Line ending conversion will be available for the active document via the Edit menu, with the selected format written to the file on the next save operation.
23. The application will maintain a configurable recent files list, defaulting to the last 10 opened files, accessible from the File menu and persisted across sessions in the XML configuration.
24. Bookmarks will be toggled on individual lines via the margin gutter or a keyboard shortcut and will be navigable forwards and backwards using dedicated Next Bookmark and Previous Bookmark commands.
25. The toolbar will contain icon buttons mapped one-to-one to their corresponding menu commands and rendered using icon sets loaded at startup; the sequential order of buttons from left to right will be: New, Open, Save, Save All, Close, Close All, Print, Cut, Copy, Paste, Undo, Redo, Find, Replace, Zoom In, Zoom Out, Synchronize Vertical Scrolling, Synchronize Horizontal Scrolling, Word Wrap, Show All Characters, Define Your Language, Document Map, Document List, Function List, Folder as Workspace, Monitoring, Start Recording, Stop Recording, and Playback.
26. The File menu's Load Session and Save Session commands will serialize and deserialize all open buffer file paths, per-file caret positions, scroll offsets, fold states, and bookmarks into an XML session file stored in the application's configuration directory.
27. The EOL Conversion feature will expose standard line ending formats and apply the chosen format to the active buffer, writing it to disk on the next explicit save.

28. The Find in Files command will utilize a background ScintillaEditView instance to scan files without loading them into visible tabs, outputting results to a dedicated panel.
29. The Monitoring command will actively poll the underlying file on disk and automatically append new data to the buffer, scrolling to the end automatically.
30. The View menu's Synchronize Vertical Scrolling and Synchronize Horizontal Scrolling commands will link the scroll positions so that scrolling either pane advances both simultaneously in the corresponding axis.
31. The Encoding menu's conversion commands will re-interpret the in-memory byte content of the active buffer under the target encoding and update the related field of the associated Buffer object, with the new encoding written to disk on the next save.
32. The Language menu's alphabetical groupings (A through Z) each expand to a submenu listing the specific language names whose display name begins with that letter, and selecting any entry invokes the corresponding Lexilla lexer module immediately without closing or reopening the file.
33. The Settings menu's Shortcut Mapper will present every Notepad++ internal command, Scintilla built-in command, plugin command, and macro shortcut in a single unified table, allowing any entry to be rebound or cleared and saving changes to the XML shortcuts configuration file.
34. The Tools menu's hash functions (MD5, SHA-1, SHA-256, SHA-512) will compute the digest of the currently selected text or, when no selection exists, the entire active document content, and display the hexadecimal result string in a dedicated dialog from which it can be copied to the clipboard.
35. The Macro menu's Start Recording and Stop Recording commands will capture the complete sequence of Scintilla commands and keystrokes issued during the recording session; the captured sequence will be stored as a named XML macro entry in the configuration file and replayed verbatim via the Playback command or the Run a Macro Multiple Times dialog.
36. The Macro menu's Trim Trailing Space and Save command will iterate over every line in the active Scintilla buffer, remove all trailing whitespace characters using inline editing operations, and then invoke the standard file save routine without prompting the user.
37. The Run menu's quick-launch entries will be stored as configurable shortcut-to-command-string mappings in the XML configuration, with the currently active file's full path substituted at runtime via the a built-in variable before the external process is launched.
38. The Help menu's Debug Info dialog will display at runtime: the Notepad++ version string, the linked Scintilla version, the linked Lexilla version, the host Windows OS version and build number, and the names of all currently loaded plugin DLLs sourced from the PluginsManager, rather than from any static configuration file.